

Multimodal Visual Positioning Application

Multimodal Visual Positioning Application

1. Concept Introduction
 - 1.1 What is "Multimodal Visual Positioning"?
 - 1.2 Implementation Principles Overview
2. Code Analysis
 - Key Code
 1. Tool Layer Entry (`largetmodel/utis/tools_manager.py`)
 2. Model Interface Layer (`largetmodel/utis/large_model_interface.py`)
 - Code Analysis
3. Practical Operations
 - 3.1 Configure Online LLM
 - 3.2 Start and Test the Function

1. Concept Introduction

1.1 What is "Multimodal Visual Positioning"?

Multimodal Visual Positioning is a technology that combines multiple sensor inputs (such as cameras, depth sensors, IMU, etc.) and algorithmic processing techniques to achieve precise position and posture recognition and tracking of devices or users in the environment. This technology does not rely on a single type of sensor data but integrates information from different perceptual modalities, thereby improving positioning accuracy and robustness.

1.2 Implementation Principles Overview

1. **Cross-modal Representation Learning:** In order to enable LLMs to handle visual information, a mechanism needs to be developed to convert visual signals into a form that models can understand. This may involve using convolutional neural networks (CNNs) or other architectures suitable for image processing to extract features and map them into the same embedding space as text.
2. **Joint Training:** By designing appropriate loss functions, text and visual data can be trained simultaneously within the same framework, allowing the model to learn to establish connections between these two modalities. For example, in a question-answering system, answers can be given based on both the provided text questions and related image content.
3. **Vision-guided Language Generation/Understanding:** Once an effective cross-modal representation is established, visual information can be leveraged to enhance the capabilities of language models. For instance, when given a photo, the model can not only describe what is happening in the image but also answer specific questions about the scene, or even execute instructions based on visual cues (such as navigating to a certain location).

2. Code Analysis

Key Code

1. Tool Layer Entry (`largemodel/utils/tools_manager.py`)

The `visual_positioning` function in this file defines the execution flow of this tool, especially how it constructs a Prompt containing the target object name and format requirements.

```
# From largemodel/utils/tools_manager.py
class ToolsManager:
    # ...
    def visual_positioning(self, args):
        """
        Locate object coordinates in image and save results to MD file.

        :param args: Arguments containing image path and object name.
        :return: Dictionary with file path and coordinate data.
        """
        self.node.get_logger().info(f"Executing visual_positioning() tool with args:
{args}")
        try:
            image_path = args.get("image_path")
            object_name = args.get("object_name")
            # ... (Path fallback mechanism and parameter checking)

            # Construct a prompt asking the large model to identify the coordinates of the
            specified object
            if self.node.language == 'zh':
                prompt = f"Please carefully analyze this image and locate every
{object_name} with bounding boxes..."
            else:
                prompt = f"Please carefully analyze this image and find the position of all
{object_name}..."

            # ... (Build independent message context)

            result = self.node.model_client.infer_with_image(image_path, prompt,
message=message_to_use)

            # ... (Process and parse returned coordinate text)

            return {
                "file_path": md_file_path,
                "coordinates_content": coordinates_content,
                "explanation_content": explanation_content
            }
        # ... (Error handling)
```

2. Model Interface Layer (`largemodel/utils/large_model_interface.py`)

The `infer_with_image` function in this file is the unified entry point for all image-related tasks.

```
# From largemodel/utils/large_model_interface.py

class model_interface:
    # ...
    def infer_with_image(self, image_path, text=None, message=None):
        """Unified image inference interface"""
        # ... (Prepare message)
        try:
            # Determine which specific implementation to call based on self.llm_platform
            if self.llm_platform == 'ollama':
                response_content = self.ollama_infer(self.messages, image_path=image_path)
            elif self.llm_platform == 'tongyi':
                # ... Call Tongyi model logic
                pass
            # ... (Other platform logic)
        # ...
        return {'response': response_content, 'messages': self.messages.copy()}
```

Code Analysis

The core of the visual positioning function lies in **guiding the large model to output structured data through precise instructions**. It also follows the layered design of the tool layer and model interface layer.

1. Tool Layer (`tools_manager.py`):

- The `visual_positioning` function is the business core of this function. It receives two key parameters: `image_path` (image path) and `object_name` (name of the object to be positioned).
- The most core operation of this function is **constructing a highly customized Prompt**. It not only simply asks the model to describe the image but embeds `object_name` into a carefully designed template, explicitly requiring the model to "locate the position of each {object_name} in the image", and implicitly or explicitly requiring the result to be returned in a specific format (such as a coordinate array).
- After constructing the Prompt, it calls the `infer_with_image` method of the model interface layer, passing the image and this customized instruction together.
- After receiving the returned text from the model interface layer, it also needs to perform **post-processing**: using regular expressions and other methods to parse precise coordinate data from the model's natural language response.
- Finally, it returns the parsed structured coordinate data to the upper-layer application.

2. Model Interface Layer (`large_model_interface.py`):

- The `infer_with_image` function continues to act as the "dispatch center". It receives the image and Prompt from `visual_positioning`, and dispatches the task to the correct backend model implementation based on the current configuration (`self.llm_platform`).

- For visual positioning tasks, the responsibility of the model interface layer is basically the same as when handling visual understanding tasks: packaging the image data and text instructions correctly, sending them to the selected model platform, and then returning the returned text result unchanged to the tool layer. All platform-specific implementation details are encapsulated in this layer.

In summary, the general flow of visual positioning is: `ToolsManager` receives the target object name and constructs a precise Prompt requiring coordinate return -> `ToolsManager` calls the model interface -> `model_interface` packages the image and Prompt together and sends them to the corresponding model platform based on the configuration -> The model returns text containing coordinate information -> `model_interface` returns the text to `ToolsManager` -> `ToolsManager` parses the text, extracts structured coordinate data and returns. This process demonstrates how Prompt Engineering technology can be used to enable a general visual large model to complete more specific and structured tasks.

3. Practical Operations

3.1 Configure Online LLM

1. Update the key in the configuration file, open the model interface configuration file

`large_model_interface.yaml`:

```
vim ~/yahboomcar_ws/src/largemodel/config/large_model_interface.yaml
```

2. Fill in your API Key:

Find the corresponding section and paste the API Key you just copied. Here taking Qwen configuration as an example

```
# large_model_interface.yaml

## Qwen
qianwen_api_key: "sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx" # Paste your Key
qianwen_model: "qwen-vl-max-latest" # You can choose models as needed, such as qwen-
turbo, qwen-plus
```

3. Open the main configuration file `yahboom.yaml`:

```
vim ~/yahboomcar_ws/src/largemodel/config/yahboom.yaml
```

4. Select the online platform to use:

Modify the `llm_platform` parameter to the platform name you want to use

```
# yahboom.yaml

model_service:
  ros__parameters:
    # ...
    llm_platform: 'tongyi' # Optional platforms: 'tongyi', 'spark', 'qianfan',
'openrouter'
```

5. When using the voice module for broadcast, check if a device is occupying the voice module. You can first enter

```
sunrise@ubuntu:~/Desktop$ arecord -l
**** List of CAPTURE Hardware Devices ****
card 0: duplexaudioi2s1 [duplex-audio-i2s1], device 0: i2s1-voicehat-hifi voiceh
at-hifi-0 [i2s1-voicehat-hifi voicehat-hifi-0]
  Subdevices: 0/1
  Subdevice #0: subdevice #0
card 1: duplexaudio [duplex-audio], device 0: i2s0-(null) ES8326 HiFi-0 [i2s0-(n
ull) ES8326 HiFi-0]
  Subdevices: 1/1
  Subdevice #0: subdevice #0
```

If the output is as shown in the image above, you can directly use the broadcast device number without modification. If different, open the configuration file

```
vim ~/yahboomcar_ws/src/largemodel/largemodel/model_service.py
```

Find the location in the image and modify it to plughw:1,0 or hw:1,0, then save.

```
472         except Exception as e:
473             self.get_logger().error(f"❌ Error in TTS callback: {e}")
474             # 出错时恢复麦克风
475             self.publish_wakeup_signal(False)
476
477     def play_tts_audio(self):
478         """播放TTS音频文件"""
479         try:
480             if not os.path.exists(self.tts_out_path):
481                 self.get_logger().error(f"❌ Audio file not found: {self.tts_out_path}")
482                 return
483
484             # 使用aplay命令播放
485             self.get_logger().info(f"🔊 Playing audio: aplay -D plughw:0,0 {self.tts_out_path}")
486
487             result = subprocess.run(
488                 ["mpg123", "-a", "plughw:1,0", self.tts_out_path],
489                 capture_output=True,
490                 text=False
491             )
492
493             if result.returncode == 0:
494                 self.get_logger().info("✅ TTS playback completed successfully")
495             else:
496                 self.get_logger().error(f"❌ TTS playback failed: {result.stderr}")
497
498         except Exception as e:
499             self.get_logger().error(f"❌ Error playing TTS audio: {e}")
500
```

After modifying the configuration file, you need to recompile in the workspace and source:

```
cd ~/yahboomcar_ws
colcon build && source install/setup.bash
```

3.2 Start and Test the Function

1. Open a new terminal and start the `audio_io` main program:

Run the following command to enable the microphone and wake-up program:

```
ros2 launch audio_io audio_io_yahboom.launch.py
```

2. Open a new terminal and start the `largemodel` main program:

Run the following command to enable voice interaction:

```
ros2 launch largemodel largemodel_control.launch.py
```

3. Test:

- **Wake-up:** Say to the microphone: "Hello, DIGUA."
- **Dialogue:** After the speaker responds, you can say: `Take a photo and analyze the position of the bottles in the image`
- **Observe logs:** In the terminal running the `launch` file, you should be able to see:
 1. The ASR node recognizes your question and prints it out.
 2. The `model_service` node receives the text, calls the LLM, and prints out the LLM's response.
- **Listen to the answer:** After a while, you should hear the answer from the speaker, and you should find an MD document recording the coordinate positions and located object information at the path `/home/sunrise/yahboomcar_ws/src/largemodel/resources_file/captured_images/captured_frame_1775198369172_position.md`.